# 👌 XAI Lecture 22

## Compiled Transformers as a Laboratory for Interpretability

FAMAF - UNC

First Semester 2026

# Disclaimer ---

**This course is based on**

**Explainable Artificial Intelligence**

From Simple Predictors to Complex Generative Models

Spring 2023, Harvard University

https://interpretable-ml-class.github.io/

# Tracr: Compiled Transformers as a Laboratory for Interpretability

**David Lindner**[†] Google DeepMind
**János Kramár** Google DeepMind
**Sebastian Farquhar** Google DeepMind
**Matthew Rahtz** Google DeepMind
**Thomas McGrath** Google DeepMind
**Vladimir Mikulik**[†] Google DeepMind

**Abstract**

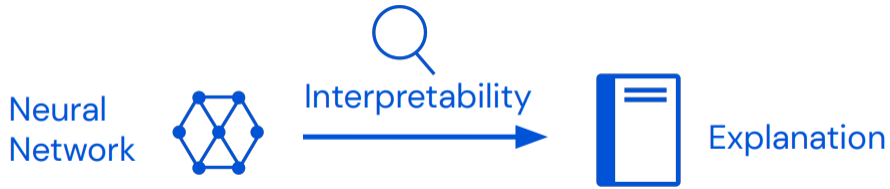
We show how to "compile" human-readable programs into standard decoder-only transformer models. Our compiler, `Tracr`, generates models with known structure. This structure can be used to design experiments. For example, we use it to study "superposition" in transformers that execute multi-step al-

(Lindner et al. 2023)

# Motivations - Tracr ---

- **Controlled experiments:** Creates simple, controlled Transformer models from RASP programs.
- **Ground truth availability:** Provides known internal mechanisms to evaluate interpretability methods.
- **Bridge algorithms $\leftrightarrow$ Transformers:** Shows how algorithms can be implemented inside attention-based models.
- **Reduction of confounding factors:** Avoids training noise and unclear learned objectives.
- **Educational and research utility:** Works as a laboratory for mechanistic interpretability.

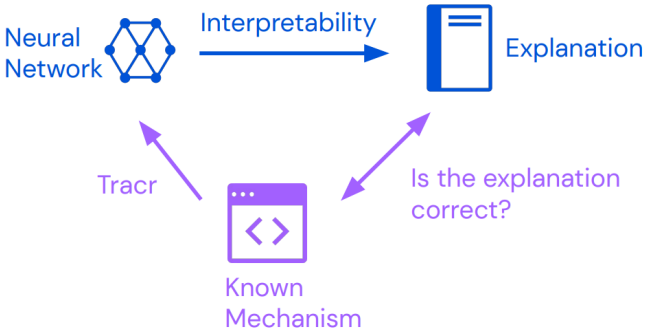
# The Core Problem: No Ground Truth ---




**Interpretability bottleneck**

For a trained neural network, there is usually no independent ground-truth mechanism telling us whether an explanation is correct.

# What If We Had Ground Truth? ---




If the network is compiled from a **known mechanism**, then an explanation can be checked against the program that generated the weights.

# Introducing Tracr ---

Known Mechanism ⟶ Neural Network

Tracr is a compiler from **RASP programs** into **decoder-only transformer weights**.

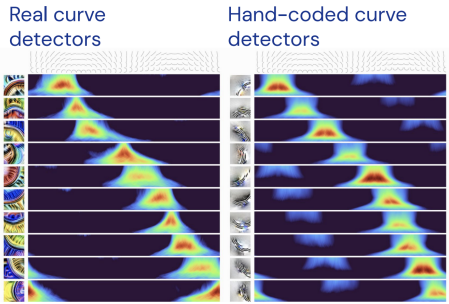
(Lindner et al. 2023)

# Plan for Today ---

1. Building a **compiler** for transformer models

2. Studying **superposition** in compiled models

# Why Not Hand-Code Weights? ---




(Cammarata et al. 2021)

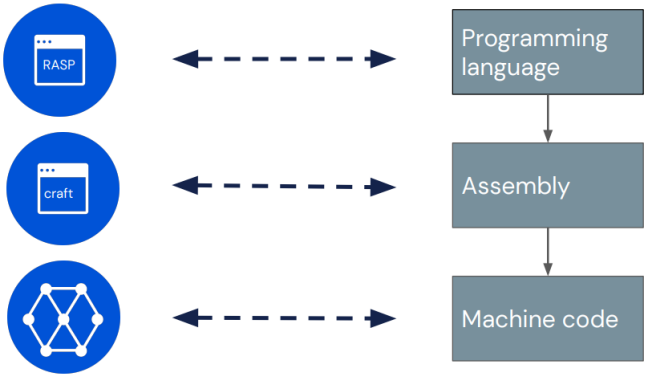
Hand-coding weights is a strong form of understanding:

- We specify the circuit directly
- We know what each component should do
- We can compare learned behavior against the construction

**Problem**

This does not scale. It is like programming a model in bytecode.

# Part 1: Building the Compiler ---




Tracr works analogously to how we would translate a programming language into executable code.

# Three-Step Translation ---


**Human readable code in domain-specific language**


**Basis independent representation of vector spaces and transformers**

**Neural network weights**

## Design principle

Keep the high-level algorithm explicit as long as possible, then lower it into a standard transformer implementation.

# RASP: A Language for Transformer Computations ---

**Definition**

**RASP** = "Restricted Access Sequence Processing Language"


- Two variable types: **sequence operations** and **selectors**
- Two instruction types: **elementwise** and **select-aggregate**
- S-ops roughly correspond to transformer **residual stream state**

**Example sequence**

bos x a c x

**Primitive s-ops**

`tokens("hello")` $= [h, e, l, l, o]$

`indices("hello")` $= [0, 1, 2, 3, 4]$

# RASP Instructions Map to Transformer Blocks ---

## Elementwise operations

- Apply a function independently at each sequence position
- Example: `(3 * indices)("hello")` $= [0, 3, 6, 9, 12]$
- Roughly correspond to transformer **MLP layers**

## Select-aggregate operations

- Move information between token positions
- A selector evaluates to an $N \times N$ binary matrix
- `select` builds the selector; `aggregate` reads selected values
- Roughly correspond to transformer **attention**

## Compiler intuition

RASP keeps the algorithm readable, while Tracr lowers these operations into transformer components.

# RASP Example: Count Previous x Tokens ---

## RASP code

```
is_x = (tokens == "x")
prevs = select(indices, indices, <=)
frac_prev = aggregate(prevs, is_x)
```

```
seq := ["a", "x", "b", "x", "c"]
tokens(seq) = ["a", "x", "b", "x", "c"]
indices(seq) = [0, 1, 2, 3, 4]

is_x(seq) = [0, 1, 0, 1, 0]
prevs(seq) = [[1, 0, 0, 0, 0],
              [1, 1, 0, 0, 0],
              [1, 1, 1, 0, 0],
              [1, 1, 1, 1, 0],
              [1, 1, 1, 1, 1]]
frac_prev(seq) = [0, 1/2, 1/3, 2/4, 2/5]
```

## In plain English

For each position, ask: *among all previous positions including this one, what fraction contained the token x?*

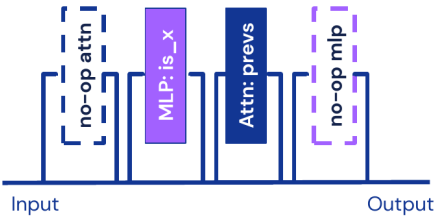
# From RASP to transformer weights ---

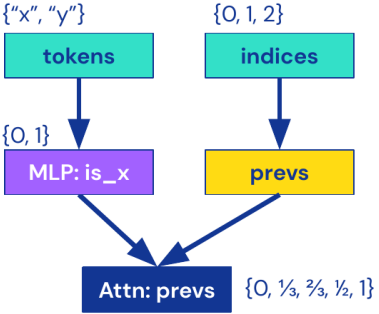
## Compilation process (6 steps)

1. Construct computational graph.
2. Infer s-op input/output values.
3. Translate s-ops into model blocks.
4. Assign components to layers.
5. Assemble the craft transformer.
6. Generate weight matrices.

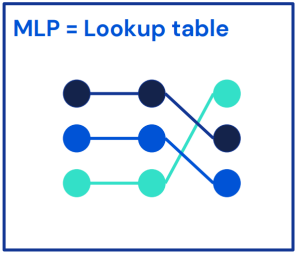
## RASP code

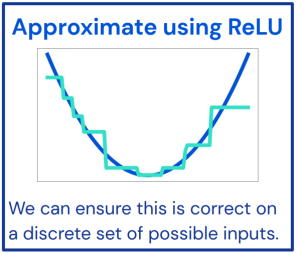
```
is_x = (tokens == "x")
prevs = select(indices, indices, <=)
frac_prev = aggregate(prevs, is_x)
```



# Implementing MLP Layers ---




## Categorical variables

An MLP can implement a lookup table over finitely many categories.

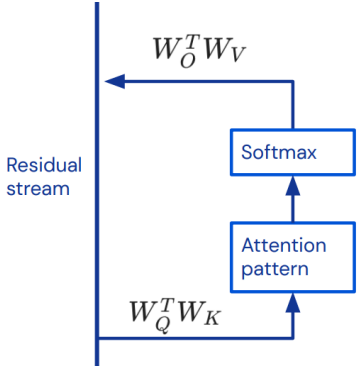
## Numerical variables

A ReLU network can approximate the desired function exactly on the discrete input set used by the program.

# Implementing Attention Heads ---

## Operation mapping

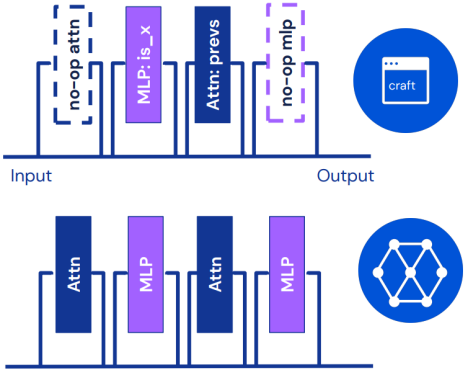
| RASP operation | Transformer component |
|---|---|
| `select(indices, indices, <=)` | query key scores $W_Q^T W_K$ |
| `aggregate(prevs, is_x)` | value output map $W_O^T W_V$ |




**In plain English**

Attention is used as programmable routing: decide *which positions to read from*, then aggregate their values.

# craft Models Map to Standard Transformers ---




## Key idea

craft separates the abstract vector-space computation from implementation details, then maps it into a GPT-like transformer architecture.

# What Can Tracr Compile? ---

**Examples**

- Count tokens and build histograms
- Detect all occurrences of a pattern
- Sort a sequence
- Check balanced parentheses
- ...

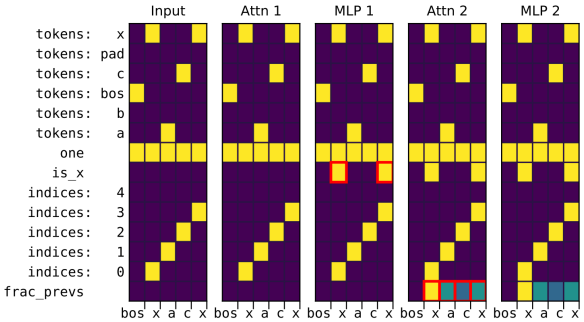
**Limitations**

- Binary attention patterns
- Algorithmic, not probabilistic, tasks
- Programs close to transformer structure
- Large and inefficient compiled models

# A Compiled Residual Stream ---

## RASP program

```
is_x = (tokens == "x")
prevs = select(indices, indices, <=)
frac_prev = aggregate(prevs, is_x)
```




## Key idea

The residual stream exposes the compiled variables: first the MLP computes `is_x`, then attention computes the fraction over previous positions.

# What Does Ground Truth Buy Us? ---

With a compiled model, we know:

- Which variables should exist
- Which layer computes each variable
- Which attention head implements each selector
- Which features are necessary for the final output

**Evaluation idea**

Run an interpretability method and ask whether it recovers the same components the compiler inserted.

**Caveat**

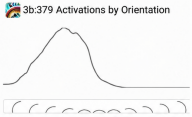
This only tests interpretability on synthetic algorithmic models, not arbitrary trained LLMs.

2. Studying **superposition** in compiled models

# The Superposition Hypothesis ---

## Observation 1

Some neurons correspond to clean, interpretable features.




Cammarata, et al., "Thread: Circuits", Distill, 2020.

## Observation 2

Other neurons appear to represent multiple features.

Simple Optimization Dataset examples

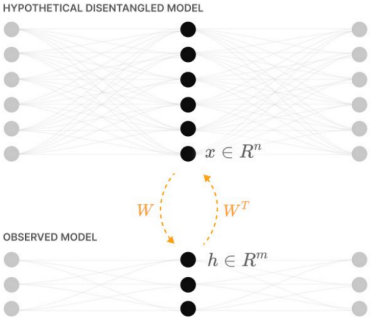
Olah, et al., "Feature Visualization", Distill, 2017.

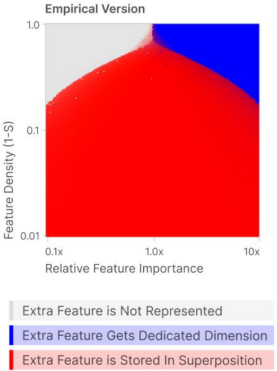
## Observation 3

A linear representation can store more features than dimensions when features are sparse.


Elhage, et al., "Toy Models of Superposition", Transformer Circuits Thread, 2022.

# Toy Models Show When Superposition Appears ---




$$h = Wx$$
$$x' = \text{ReLU}(W^T h + b)$$
$$x' = \text{ReLU}(W^T W x + b)$$

$$L = \sum_x \sum_i I_i (x_i - x'_i)^2$$

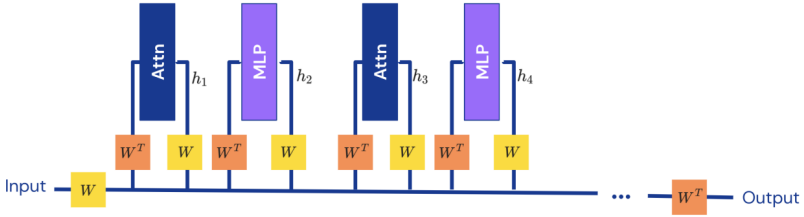
(Elhage et al., "Toy Models of Superposition", Transformer Circuits Thread, 2022.)

# Why Compress Tracr Models? ---

| In toy models we see superposition if | | In Tracr models |
|---|---|---|
| 1. Features are **sparse** | $\Longrightarrow$ | 1. Features are **sparse** |
| 2. Some features are more **important** than others | $\Longrightarrow$ | 2. Some features are more **important** for the computation |
| 3. The model has to use **fewer dimensions than features** | $\Longrightarrow$ | 3. Can we **compress** the model to use **fewer dimensions**? |

Tracr models already have sparse, meaningful variables. If we force their residual stream to use fewer dimensions, we can study superposition with a known feature basis.

# Linear Compression Objective ---




We train $W \in \mathbb{R}^{D \times d}$ to minimize:

$$\mathcal{L}_{\text{total}}(W) = \mathbb{E}_x \left[ \mathcal{L}_{\text{out}}(W, x) + \mathcal{L}_{\text{layer}}(W, x) \right]$$

$$\mathcal{L}_{\text{out}}(W, x) = \text{loss}\left( f(x), \hat{f}_W(x) \right)$$

**minimize output loss**

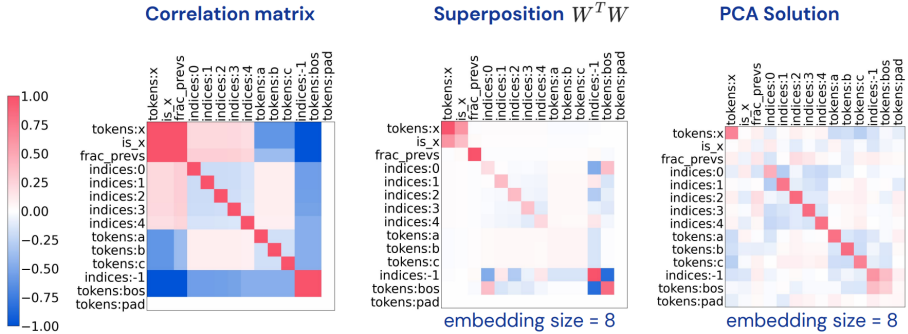
$$\mathcal{L}_{\text{layer}}(W, x) = \sum_{\text{layer } i} \left( h_i(x) - \hat{h}_{W,i}(x) \right)^2$$

**implement the same computation**

## In plain English

Train only a compression map $W$: the compressed model should preserve the final output and keep each intermediate layer close to the original compiled computation.

# What Changes After Compression? ---

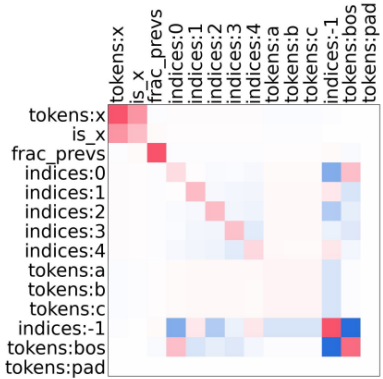



The learned $W^TW$ structure is qualitatively different from PCA: less important and more independent features are packed into shared directions.

# Which Features Enter Superposition? ---

The lecture highlights three drivers:

- **Importance:** how much the computation needs the feature
- **Density:** how often the feature is active
- **Linear independence:** how easily it can share directions with others



# Discussion Questions ---

- **Can Tracr create evaluation benchmarks** for interpretability tools?
- **Can we reverse superposition** in Tracr models?
  *Sparse coding, dictionary learning*
- **Can we manually replace model components** we think we understand?

# Summary ---

- Tracr compiles RASP programs into transformer weights
- The compiled model has a known mechanism
- This gives interpretability methods an answer key
- Compression induces superposition in a controlled feature basis
- The main value is experimental control, not realism

**Resources**

https://github.com/google-deepmind/tracr

https://arxiv.org/abs/2301.05062

# Thank You!

# References ---


Lindner, David, János Kramár, Sebastian Farquhar, Matthew Rahtz, Thomas McGrath, and Vladimir Mikulik. 2023. "Tracr: Compiled Transformers as a Laboratory for Interpretability." arXiv:2301.05062.

Weiss, Gail, Yoav Goldberg, and Eran Yahav. 2021. "Thinking Like Transformers." International Conference on Machine Learning.

Cammarata, Nick, Shan Carter, Gabriel Goh, Chris Olah, Michael Petrov, Ludwig Schubert, Chelsea Voss, Ben Egan, and Swee Kiat Lim. 2020. "Thread: Circuits." Distill.

Cammarata, Nick, Shan Carter, Gabriel Goh, Chris Olah, Michael Petrov, Ludwig Schubert, and Chelsea Voss. 2021. "Curve Circuits." Distill.

Olah, Chris, Alexander Mordvintsev, and Ludwig Schubert. 2017. "Feature Visualization." Distill.

Elhage, Nelson, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, and others. 2022. "Toy Models of Superposition." Transformer Circuits Thread.